

Състезателна Бесеница

Документация на проекта - Операционни системи, Вариант 3

hangman-server.c | hangman-client.c | game.c / game.h | Makefile

1. Въведение

Проектът реализира мрежова игра Състезателна бесеница с архитектура клиент-сървър. Два клиента се свързват към общ сървър - всеки подава дума, която опонентът трябва да отгатне. Печели играчът с по-малко грешни букви. При равен брой - резултатът е равенство.

- **Игра:** Двама играчи избират дума, която другият трябва да отгатне.
- **Технологии:** Написано на C, TCP сокети, многонишковост с POSIX pthreads.
- **Архитектура:** Един сървър обслужва точно 2 клиента едновременно.
- **Файлове:** game.h / game.c (предоставени, непроменени) + hangman-server.c + hangman-client.c + Makefile

2. Комуникационен протокол

Протоколът е текстов, базиран на редове (\n). Всяко съобщение е един ред. Комуникацията е двупосочна по TCP сокет.

2.1 Съобщения клиент -> сървър

Съобщение	Описание
WORD <дума>	Подава думата, която опонентът трябва да отгатне
GUESS <буква>	Познава буква от думата, която клиентът отгатва

2.2 Съобщения сървър -> клиент

Съобщение	Описание
WORD_OK	Думата е приета успешно
WORD_ERR	Невалидна дума - сървърът затваря връзката
STATE <pat> <cnt> <inc>	Текущо състояние: маскирана дума, брой грешки, грешни букви
STATE_SOLVED <pat> <cnt> <inc>	Думата е решена - чакайте RESULT
RESULT <W/L/T> <my> <opp>	Краен резултат с грешните букви на двамата

2.3 Пример за пълна игра

Клиент А подава 'cat', Клиент В подава 'dog'. А познава 'dog' без грешки, В прави 1 грешка (буква x):

```
Клиент А          Сървър          Клиент В
-----
WORD cat  ----->
          <----- WORD_OK
                                <----- WORD dog
                                WORD_OK ----->
                                [sw[0]='dog', sw[1]='cat']
<----- STATE ____ 0
                                STATE ____ 0 ----->
GUESS d  ----->
<----- STATE d__ 0
                                <----- GUESS x
                                STATE ____ 1 x ----->
GUESS o, g -----> ...
<----- STATE_SOLVED dog 0
          [А чака В да завърши]
                                STATE_SOLVED cat 1 x ----->
<----- RESULT W |x
                                RESULT L x| ----->
```

3. Сървър (hangman-server.c)

Сървърът се стартира с `./hangman-server <port>`. Слуша на 0.0.0.0 и приема точно 2 клиента, след което спира да приема нови връзки. За всеки клиент се стартира отделна POSIX нишка.

3.1 Глобална структура `game_t`

Цялото игрово състояние е инкапсулирано в единна статична структура, споделена между двете нишки:

```
typedef struct {
    int fd[2];           // файлови дескриптори на клиентите
    char word[2][256];   // думите, подадени от всеки клиент
    secret_word_t sw[2]; // игровото състояние за всеки играч
    int solved[2];       // флаг: дали играчът е решил думата
    int words_received;  // брой получени думи (0, 1 или 2)
    int phase;           // 0 = чакаме думи, 1 = играем
    pthread_mutex_t mu;
    pthread_cond_t cond;
} game_t;

static game_t G;
```

3.2 main() - инициализация и приемане на клиенти

Сокетът се свързва с INADDR_ANY (0.0.0.0). SO_REUSEADDR позволява незабавен рестарт:

```
int srv = socket(AF_INET, SOCK_STREAM, 0);
setsockopt(srv, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
addr.sin_addr.s_addr = INADDR_ANY;
bind(srv, ...); listen(srv, 5);
printf("Listening on %d...\n", port);

for (int i = 0; i < 2; i++) {
    int cfd = accept(srv, ...);
    G.fd[i] = cfd;
}
close(srv); // повече връзки не се приемат

pthread_create(&t[0], NULL, player_thread, (void*)(intptr_t)0);
pthread_create(&t[1], NULL, player_thread, (void*)(intptr_t)1);
pthread_join(t[0], NULL);
pthread_join(t[1], NULL);
```

3.3 player_thread() - валидация и кръстосано присвояване

Всяка нишка чете WORD и валидира чрез secret_word_init_from_c_string върху временна структура. Ако думата съдържа не-букви или е празна - изпраща WORD_ERR и прекратява:

```
secret_word_t tmp_sw;
if (!secret_word_init_from_c_string(&tmp_sw, w)) {
    send_line(fd, "WORD_ERR");
    goto done;
}
secret_word_free(&tmp_sw);
send_line(fd, "WORD_OK");
```

Когато и двата клиента са изпратили думи, се прави кръстосано присвояване - sw[0] получава word[1] и обратно. Сървърът никога не изпраща думата обратно на клиента, избрал я, което предотвратява мамене:

```
if (G.words_received == 2) {
    secret_word_init_from_c_string(&G.sw[0], G.word[1]); // A отгатва думата на B
    secret_word_init_from_c_string(&G.sw[1], G.word[0]); // B отгатва думата на A
    G.phase = 1;
    pthread_cond_broadcast(&G.cond);
} else {
    while (G.phase == 0)
        pthread_cond_wait(&G.cond, &G.mu);
}
```

3.4 Игрови цикъл

При всяко GUESS нишката обработва буквата и изпраща актуализирано STATE. build_pattern() изгражда маскирания модел:

```
char guess = line[6];
pthread_mutex_lock(&G.mu);
secret_word_guess(&G.sw[idx], guess);

int is_solved = secret_word_is_solved(&G.sw[idx]);
send_state(idx, is_solved); // STATE или STATE_SOLVED
pthread_mutex_unlock(&G.mu);

// build_pattern: разкрити букви показва, скрити -> '_'
for (size_t i = 0; i < sw->word_length; i++) {
    char ch;
    out[i] = (secret_word_letter_at(sw, i, &ch)
              == SECRET_WORD_LETTER_REVEALED) ? ch : '_';
}
```

3.5 Синхронизация при край

Нишката, решила думата, блокира на condition variable докато опонентът не завърши. Едва тогава и двамата получават RESULT:

```
if (is_solved) {
    G.solved[idx] = 1;
    pthread_cond_broadcast(&G.cond);
    while (!G.solved[1 - idx])
        pthread_cond_wait(&G.cond, &G.mu);

    char result_char = (my_err < opp_err) ? 'W' :
                      (my_err > opp_err) ? 'L' : 'T';
    snprintf(rmsg, sizeof(rmsg), "RESULT %c %s|%s",
             result_char, my_inc, opp_inc);
    send_line(fd, rmsg);
}
```

4. Клиент (hangman-client.c)

Клиентът се стартира с ./hangman-client <host> <port> <дума>. Веднага след свързване изпраща думата за опонента.

4.1 Свързване

Използва getaddrinfo за резолване - поддържа IPv4 и IPv6 прозрачно:

```
struct addrinfo hints = {0}, *res;
hints.ai_family = AF_UNSPEC; // IPv4 или IPv6
```

```

hints.ai_socktype = SOCK_STREAM;
getaddrinfo(host, portstr, &hints, &res);
int fd = socket(res->ai_family, res->ai_socktype, res->ai_protocol);
connect(fd, res->ai_addr, res->ai_addrlen);
freeaddrinfo(res);

snprintf(msg, sizeof(msg), "WORD %s", word);
send_line(fd, msg);

```

4.2 Игрови цикъл и нормализиране на буква

При STATE клиентът показва играта и чете буква от stdin. При STATE_SOLVED само изчаква RESULT:

```

while (1) {
    read_line(fd, line, sizeof(line));

    if (strncmp(line, "STATE ", 6) == 0 ||
        strncmp(line, "STATE_SOLVED ", 13) == 0) {
        parse_and_show_state(line, &solved);
        if (solved) continue;    // само чакаме RESULT

        char guess = 0;
        while (guess == 0) {
            int c = fgetc(stdin);
            if (c >= 'A' && c <= 'Z') c = c - 'A' + 'a'; // нормализация
            if (c >= 'a' && c <= 'z') guess = (char)c;
            // пропуска Enter, интервали и всичко друго
        }
        send_line(fd, gmsg);    // GUESS <буква>

    } else if (strncmp(line, "RESULT ", 7) == 0) {
        break;
    }
}

```

4.3 Показване на резултата

Форматът RESULT W my_inc|opp_inc се разпарсва чрез разделителя '|':

```

char outcome = *p;    // 'W', 'L' или 'T'
p += 2;

char *sep = strchr(p, '|');
if (sep) {
    strncpy(my_inc, p, sep - p);
    strncpy(opp_inc, sep+1, sizeof(opp_inc)-1);
}

if (outcome == 'W') printf("YOU WIN! :) \n");

```

```

else if (outcome == 'L') printf("You Lose! :(\n");
else                    printf("Tie :/\n");
printf("Your incorrect guesses: %s\n",      my_inc);
printf("Opponent's incorrect guesses: %s\n", opp_inc);

```

5. Библиотека game.h / game.c

Файловете са предоставени и не са модифицирани по никакъв начин.

5.1 Структури

```

// Битова маска: 'a'=бит0, 'b'=бит1, ..., 'z'=бит25
typedef struct letter_set {
    uint32_t bitfield;
} letter_set_t;

typedef struct secret_word {
    char      *word;                // нормализирана дума (малки букви)
    uint8_t   *revealed_bitfield;  // кои позиции са разкрити
    size_t    word_length;
    letter_set_t incorrect_guesses;
    letter_set_t all_guesses;
} secret_word_t;

```

5.2 Използвани функции

Функция	Действие
secret_word_init_from_c_string()	Инициализира и валидира думата - false ако съдържа не-букви
secret_word_guess()	Обработка буква: CORRECT, INCORRECT, INVALID или ALREADY_GUESSED
secret_word_letter_at()	Връща буква на позиция ако е разкрита, иначе HIDDEN
secret_word_is_solved()	Проверява дали всички позиции са разкрити
secret_word_incorrect_guess_count()	Брой грешно познати букви - за финалния резултат
secret_word_free()	Освобождава заделената памет

6. Компилиране и тестване

6.1 Makefile

```

CC      = gcc
CFLAGS = -Wall -Wextra -g

all: hangman-server hangman-client

hangman-server: hangman-server.c game.c game.h
    $(CC) $(CFLAGS) -o hangman-server hangman-server.c game.c -lpthread

```

```
hangman-client: hangman-client.c game.h
$(CC) $(CFLAGS) -o hangman-client hangman-client.c

clean:
rm -f hangman-server hangman-client
```

6.2 Ръчен тест

```
# Компилиране
make all

# Терминал 1 - стартиране на сървъра
./hangman-server 8080
# Очакван изход: Listening on 8080...

# Терминал 2 - Клиент А (задава 'cat' за опонента)
./hangman-client 127.0.0.1 8080 cat

# Терминал 3 - Клиент В (задава 'dog' за опонента)
./hangman-client 127.0.0.1 8080 dog

# Автоматични публични тестове
python3 test_hangman_public.py
```

Забележка: Проектът е предназначен за Linux/WSL. Предупрежденията в VS Code за 'unistd.h', 'pthread.h' и др. са само IntelliSense грешки и не влияят на компилирането.